AI Drug Toxicity Prediction Lab

Complete Project Report for Presentation

Molecular Fingerprinting | Machine Learning | Confidence Scoring

Technology Stack: Python | Streamlit | RDKit | Scikit-learn | ReportLab
Dataset: Tox21 | Features: Morgan Fingerprints (ECFP4, 1024 bits)

1. Project Overview — What Is This Application?

This project is an AI-powered Drug Toxicity Prediction Lab built as a web application. Its purpose is to take a chemical compound described by its SMILES string, convert it into numerical features using molecular fingerprinting, and then use a trained Machine Learning (ML) model to predict whether that compound is Toxic or Non-Toxic.

Think of it like this: a chemist draws a molecule on paper and wants to know if it could harm a human body before spending months testing it in a laboratory. This tool gives an instant AI-powered answer in seconds — not replacing the lab, but giving an early estimate.

Use Case	Early-stage drug/chemical safety screening using Machine Learning
Dataset Used	Tox21 — a large toxicology dataset with thousands of chemical compounds
Prediction Type	Binary Classification: Toxic (1) vs Non-Toxic (0)
Input	SMILES string — a text representation of a molecule's structure
Output	Prediction label + Confidence % + Molecular Descriptors + PDF Report

2. Libraries — What Each Library Does

Every library imported in this project has a specific job. Below is a complete explanation of each one:

2.1 Streamlit (import streamlit as st)

Streamlit is the GUI (Graphical User Interface) framework that turns this Python script into a fully interactive web application. Without Streamlit, all results would only appear as text in a terminal. With Streamlit, you get:

- A beautiful browser-based interface with buttons, input boxes, dropdowns, tabs
- Real-time interaction — click Train and watch charts appear instantly
- Sidebar panels, progress bars, download buttons, tables
- All of this with almost no traditional web development (no HTML/CSS/JavaScript needed from the developer's side)

Key functions used in this code:

Streamlit Function	What It Does
st.set_page_config()	Sets the browser tab title, icon, and layout (wide mode)
st.markdown()	Injects raw HTML/CSS for custom styling and visual design
st.tabs()	Creates the three tabs: Training, Prediction, History
st.columns()	Divides the page into side-by-side columns
st.button()	Creates clickable buttons (Train & Evaluate, Predict)
st.text_input()	Input box where user types the SMILES string
st.selectbox()	Dropdown menu to choose the ML model
st.session_state	Stores data (trained model, history) between interactions
st.spinner()	Shows a loading animation during training
st.success/error/warning()	Displays colored alert messages
st.dataframe()	Displays a table of molecular descriptors
st.image()	Renders the molecule structure image
st.download_button()	Provides the PDF download button
st.pyplot()	Renders Matplotlib charts inside the app

st.bar_chart()	Quick bar chart for probability display
st.progress()	Visual confidence progress bar
st.cache_data	Caches the CSV loading so it doesn't reload every time
st.rerun()	Refreshes the app (used when clearing history)
st.stop()	Halts execution if data files are missing

2.2 Pandas (import pandas as pd)

Pandas is the data manipulation library. It is used to:

- Load the training dataset (tox21_train_clean.csv) and test dataset (ktest.csv) from CSV files
- Create and display DataFrames (tables) in the Streamlit interface
- Count predictions in the History tab (toxic vs non-toxic)
- Export the prediction history as a CSV file for download

```
train_df, test_df = load_data()  # Loads both CSV files into Pandas DataFrames
```

2.3 NumPy (import numpy as np)

NumPy is the numerical computing library that provides array operations. In this project, it supports the underlying math for the machine learning algorithms. Scikit-learn itself uses NumPy arrays internally for matrix calculations. While not called directly many times in the user-facing code, it is the computational backbone that makes fast ML operations possible.

2.4 Matplotlib (import matplotlib.pyplot as plt)

Matplotlib creates the 4-panel chart displayed after training a model. The four charts created are:

- Scatter plot — Shows actual toxicity labels for each test compound
- Bar chart — Shows class distribution (how many toxic vs non-toxic)
- Line plot — Shows the first 50 actual labels as a trend line
- Confusion Matrix heatmap — The most important chart, showing correct vs incorrect predictions

The rcParams settings at the top of the code configure the dark 'Royal Lab' theme for all charts (navy background, gold/teal colors).

2.5 RDKit (from rdkit import ...)

RDKit is the cheminformatics library — the most specialized library in this project. It is specifically designed to work with chemical structures. This is what allows the program to understand molecules.

RDKit Module/Function	What It Does in This Project
Chem.MolFromSmiles(smiles)	Parses a SMILES text string and creates a molecule object
Chem.MolToSmiles(mol)	Converts the molecule back to its canonical SMILES form
Chem.MolToInchiKey(mol)	Generates the InChIKey — a unique hash identifier for the molecule
Chem.AddHs(mol)	Adds hydrogen atoms to count total atoms including H
Chem.GetFormalCharge(mol)	Gets the overall electrical charge of the molecule
Draw.MolToImage(mol)	Renders the molecule as a 2D image (the structure diagram shown)
Descriptors.MolWt(mol)	Calculates molecular weight in g/mol
Descriptors.TPSA(mol)	Calculates Topological Polar Surface Area (drug absorption predictor)
Crippen.MolLogP(mol)	Calculates LogP (fat solubility — important for drug delivery)
Crippen.MolMR(mol)	Calculates Molar Refractivity (polarizability of molecule)
Lipinski.NumHDonors(mol)	Counts H-Bond Donors (Lipinski rule check)
Lipinski.NumHAcceptors(mol)	Counts H-Bond Acceptors (Lipinski rule check)
Lipinski.NumRotatableBonds(mol)	Counts rotatable bonds (molecular flexibility)
rdMolDescriptors.CalcMolFormula(mol)	Calculates molecular formula (e.g., C2H6O for ethanol)
rdMolDescriptors.CalcNumRings(mol)	Counts total number of rings in the molecule
rdMolDescriptors.CalcNumAromaticRings(mol)	Counts aromatic rings (like benzene rings)
GetMorganGenerator(radius=2, fpSize=1024)	Creates Morgan Fingerprint generator (ECFP4 algorithm)
morgan_gen.GetFingerprint(mol)	Converts molecule into a 1024-bit binary fingerprint vector
RDLogger.DisableLog()	Suppresses RDKit warning messages in the console

2.6 Scikit-learn (from sklearn ...)

Scikit-learn provides the three Machine Learning algorithms used in this project. It also provides the evaluation metrics.

Scikit-learn Component	Purpose
RandomForestClassifier	The Random Forest ML model (see Section 3 for full explanation)
LogisticRegression	The Logistic Regression ML model
SVC (Support Vector Classifier)	The Support Vector Machine ML model
accuracy_score(y_test, y_pred)	Calculates what fraction of predictions were correct
f1_score(y_test, y_pred)	Calculates F1 Score — balance between precision and recall
confusion_matrix(y_test, y_pred)	Creates the 2x2 matrix of TP, FP, TN, FN results

2.7 ReportLab (from reportlab ...)

ReportLab is a professional PDF generation library. It is used to create the downloadable PDF report for each molecule prediction. It is far more capable than simple HTML-to-PDF conversion.

ReportLab Component	What It Does
SimpleDocTemplate	The main PDF document object — sets page size (A4), margins, title
Paragraph	A block of text with a style (heading, body, disclaimer, mono)
Spacer	Adds vertical space between elements in the PDF
Table / TableStyle	Creates formatted tables in the PDF (identity, descriptors, Lipinski)
Image as RLImage	Embeds the molecule structure PNG image into the PDF
PageBreak	Starts a new page in the PDF
getSampleStyleSheet()	Gets default paragraph styles to build on
ParagraphStyle	Defines custom text styles (font size, color, spacing, background)
colors	Color definitions used for text and cell backgrounds
A4 (pagesize)	Standard A4 page dimensions (210x297mm)
cm (units)	Centimeter units for precise PDF layout measurements

2.8 io and datetime

These are Python standard library modules (no installation needed):

- **Creates an in-memory binary buffer — used to save the molecule PNG image and PDF into memory without saving to disk**
- **Gets the current date and time — used to timestamp predictions and name PDF files**

3. Machine Learning Models — Full Explanation

The application offers three ML models for the user to choose from. All three are classification models — meaning they put each molecule into one of two categories: Toxic (1) or Non-Toxic (0).

3.1 What is a Machine Learning Model?

Think of a machine learning model like a very smart student who studies thousands of examples of molecules with known toxicity labels (from the Tox21 dataset). After studying, it learns patterns — for example, 'molecules with this type of fingerprint pattern tend to be toxic.' Once trained, it can predict the label for a new, unseen molecule in milliseconds.

3.2 Random Forest Classifier

Random Forest is the most popular model in this type of problem and is often the best performer. Here is how it works, step by step:

- **It creates many decision trees (like flowcharts). Each tree asks a series of Yes/No questions about the fingerprint bits.**
- **Each tree is trained on a random subset of the training data (this is the 'Random' in Random Forest).**
- **Each tree votes: Toxic or Non-Toxic?**
- **The majority vote wins. If 80 out of 100 trees say Toxic, the final answer is Toxic with high confidence.**

Why is it good for this project? Because molecular fingerprints have 1024 features and many of them are irrelevant — Random Forest naturally ignores irrelevant features and focuses on the ones that matter most. It also handles class imbalance well (there are usually fewer toxic molecules than non-toxic ones in real datasets).

Feature	Random Forest Detail
Algorithm Type	Ensemble of Decision Trees
Number of Trees	100 (default in scikit-learn)
Voting	Majority vote across all trees
Probability Output	Yes — uses predict_proba() for confidence scores

Handles Imbalance	Reasonably well
Speed	Moderate — slower to train than Logistic Regression
Best For	High-dimensional, sparse data like fingerprints

3.3 Logistic Regression

Despite having 'regression' in the name, Logistic Regression is a classification algorithm. It was designed for exactly this kind of binary (yes/no, toxic/non-toxic) decision problem.

How it works: It learns a weight (importance score) for each of the 1024 fingerprint bits. A positive weight means that bit pushes toward Toxic; a negative weight pushes toward Non-Toxic. It combines all 1024 weighted bits into one score, then passes that score through a sigmoid function to produce a probability between 0% and 100%.

Feature	Logistic Regression Detail
Algorithm Type	Linear classifier with sigmoid activation
Parameters	max_iter=1000 (allows enough iterations to converge)
Probability Output	Yes — naturally outputs probabilities
Speed	Very fast to train
Interpretability	High — can inspect which bits matter most
Best For	Linearly separable data; good baseline model

3.4 SVM — Support Vector Machine (SVC)

SVM finds the best decision boundary (a hyperplane) that separates toxic from non-toxic molecules in the 1024-dimensional fingerprint space. It maximizes the margin between the two classes.

In simpler terms: imagine plotting all molecules in a space where each axis is one of the 1024 fingerprint bits. SVM draws a line (or hyperplane in high dimensions) that best separates the toxic group from the non-toxic group, keeping as much distance as possible from both groups.

Feature	SVM Detail
Algorithm Type	Kernel-based maximum margin classifier
probability=True	Required to enable predict_proba() for confidence scores
Kernel	RBF (Radial Basis Function) by default

Speed	Slowest of the three — especially on large datasets
Best For	High-dimensional data; excellent when margins are clear

3.5 Why Three Models Are Offered

Different models perform differently depending on the specific dataset. By allowing the user to choose and compare all three, the application lets researchers see which model performs best on their specific data subset. The metrics (Accuracy, F1 Score, Confusion Matrix) shown after training allow a direct comparison.

4. SMILES Strings and Morgan Fingerprints

4.1 What is SMILES?

SMILES stands for Simplified Molecular Input Line Entry System. It is a way to write a molecule's structure as a simple text string. For example:

SMILES String	Molecule	Description
CCO	Ethanol	Two carbons (CC) bonded to oxygen (O)
c1ccccc1	Benzene	Six aromatic carbons in a ring
CC(=O)Oc1ccccc1C(=O)O	Aspirin	Complex structure with multiple functional groups

The Chem.MolFromSmiles() function reads this text and creates a full molecular object with all the bonds, atoms, and structural information. If the SMILES is invalid, it returns None and the app shows an error.

4.2 What is a Morgan Fingerprint?

A machine learning model cannot directly read a molecule — it needs numbers. A Morgan Fingerprint converts the entire molecular structure into a fixed-length list of 0s and 1s (a binary vector).

How it works: Starting from each atom, the algorithm explores the local neighbourhood of the atom up to a certain radius (radius=2 in this code, which is why it's called ECFP4 — Extended Connectivity FingerPrint of diameter 4). Each unique neighbourhood pattern is hashed to one of 1024 positions in the fingerprint vector, and that position is set to 1.

```
morgan_gen = GetMorganGenerator(radius=2, fpSize=1024)
fingerprint = list(morgan_gen.GetFingerprint(mol))  # A list of 1024 zeros and ones
```

The result is a 1024-element vector like [0, 1, 0, 0, 1, 1, 0, ...] — and this is the actual input fed into the ML model. Two similar molecules will have very similar fingerprints; very different molecules will have very different fingerprints.

5. Complete Code Walkthrough — Line by Line

This section explains every major block of code in the order it appears in app_r.py.

5.1 Imports Block

```
import streamlit as st
```

Imports Streamlit as 'st' — the shorthand used throughout the code to call all Streamlit functions.

```
import pandas as pd
```

Imports Pandas as 'pd' — used for loading CSV data and creating data tables.

```
import numpy as np
```

Imports NumPy — mathematical foundation for array computations.

```
import matplotlib.pyplot as plt
```

Imports Matplotlib's plotting module — used to create the 4-panel chart.

```
from matplotlib import rcParams
```

Imports rcParams which allows global chart styling (dark navy theme).

```
from rdkit import Chem, RDLogger
```

Chem: the core RDKit molecule module. RDLogger: controls RDKit's log output.

```
from rdkit.Chem import Draw, Descriptors, Crippen, Lipinski, rdMolDescriptors
```

Draw: generates 2D molecule images. Descriptors: molecular properties. Crippen: LogP/MR. Lipinski: Rule of Five. rdMolDescriptors: advanced descriptors like formula, ring count.

```
from rdkit.Chem.rdFingerprintGenerator import GetMorganGenerator
```

Imports the Morgan fingerprint generator — the core feature extraction tool.

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
```

The three ML model classes. Only one is instantiated per training session.

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
```

Three evaluation functions to measure model performance.

```
import io
```

In-memory binary streams for handling PNG/PDF bytes without disk I/O.

```
from datetime import datetime
```

Used to timestamp predictions and create unique PDF file names.

```
from reportlab.lib.pagesizes import A4
```

Defines the A4 page size for the PDF report.

All the other reportlab imports (styles, colors, platypus elements) define the visual building blocks for constructing the PDF document programmatically.

```
RDLogger.DisableLog('rdApp.*')
```

Suppresses all RDKit internal messages/warnings so they don't clutter the terminal.

5.2 Page Configuration

```
st.set_page_config(page_title='Drug Toxicity Prediction Lab', page_icon='💊',
layout='wide', initial_sidebar_state='expanded')
```

This MUST be the first Streamlit command in the script. It:

- Sets the browser tab title and the favicon emoji icon
- Sets layout='wide' to use the full browser width instead of a narrow column
- Opens the sidebar by default (initial_sidebar_state='expanded')

5.3 Custom CSS Styling (st.markdown with unsafe_allow_html)

The large CSS block starting with <style> defines the entire visual theme of the application — called the 'Royal Lab' theme. This is injected as raw HTML into the page.

Key CSS concepts used:

- **Defines a color palette (navy, gold, teal, red, green) as CSS variables reused throughout all styling**

- **The frosted glass card effect using backdrop-filter: blur() and semi-transparent backgrounds**
- **translateY(-4px) and box-shadow changes on hover give the cards a lifting effect**
- **Loads Playfair Display (serif headers), Inter (body text), and JetBrains Mono (code/metrics)**
- **Creates the complex background with three overlapping gradient glows**
- **The Toxic/Safe badge slowly pulses brighter and dimmer**

5.4 Matplotlib Dark Theme Configuration

```
rcParams['axes.facecolor'] = '#131c36'
```

Sets the background color of chart axes to match the dark navy theme. All rcParams settings configure Matplotlib's global defaults so that every chart generated uses the same color scheme without needing to set it per chart.

5.5 Session State Initialization

```
if 'history' not in st.session_state:     st.session_state['history'] = []
```

Streamlit reruns the entire script from top to bottom every time the user clicks a button. Session state is how data persists between these reruns. Here, we initialize two keys:

- **A list that accumulates all predictions made during the session**
- **Stores the most recent prediction payload so the user can re-download the PDF after navigating away**

5.6 The featurize() Function

```
def featurize(smiles):     mol = Chem.MolFromSmiles(smiles)     if mol is None:
return None     return list(morgan_gen.GetFingerprint(mol))
```

This is the feature extraction function. Every SMILES string in the dataset must be converted to a fingerprint before it can be fed to the ML model. Steps:

- **Parse the SMILES into a molecule object**
- **If parsing fails (invalid SMILES), return None to mark it as unusable**
- **Generate the 1024-bit Morgan fingerprint and return it as a Python list**

5.7 The compute_descriptors() Function

This function computes 14 chemical properties of a molecule. These are displayed in the 'Molecular Descriptors' table and included in the PDF report. Each property provides different information about the molecule:

Descriptor	What It Tells Us	Significance in Drug Design
Molecular Weight	Total mass of the molecule	Lipinski rule: must be <= 500 g/mol for oral drugs
LogP (Crippen)	Lipophilicity (fat vs water solubility)	Controls how easily drug crosses cell membranes
TPSA	Polar Surface Area	Predicts intestinal absorption and blood-brain penetration
H-Bond Donors	Count of N-H and O-H groups	Lipinski rule: must be <= 5
H-Bond Acceptors	Count of N and O atoms	Lipinski rule: must be <= 10
Rotatable Bonds	Molecular flexibility	Affects bioavailability
Rings / Aromatic Rings	Ring count	Aromatic rings often linked to toxicity
Formal Charge	Overall electric charge	Affects solubility and protein binding
Molar Refractivity	Polarizability	Related to drug potency

5.8 The lipinski_assessment() Function

Lipinski's Rule of Five (Ro5) is a famous set of guidelines from Pfizer scientist Christopher Lipinski that predicts whether a drug candidate will be orally bioavailable (able to be taken as a pill and absorbed into the bloodstream).

Rule	Threshold	Why It Matters
Molecular Weight	<= 500 g/mol	Large molecules don't absorb well through the gut
LogP	<= 5	Very fat-soluble molecules have poor water solubility
H-Bond Donors	<= 5	Too many donors means poor membrane permeability
H-Bond Acceptors	<= 10	Too many acceptors also hurts membrane crossing

If a molecule violates 2 or more of these rules, it is generally considered unlikely to be a good oral drug. The function returns the pass/fail status for each rule and the total violation count.

5.9 The build_material_pdf_report() Function

This function constructs a professional multi-section PDF report using ReportLab's 'Platypus' (Page Layout and Typography Using Scripts) system. The report includes:

- A disclaimer box — warns that AI predictions are not lab-certified
- Material Identity table — SMILES, canonical SMILES, molecular formula, InChIKey
- Structure & Prediction side-by-side — the 2D molecule image next to prediction results
- Molecular Descriptors table — all 14 computed properties
- Lipinski's Rule of Five table — with color-coded Pass/Fail

The function uses io.BytesIO() to build the entire PDF in memory (RAM) and returns it as bytes, which Streamlit's download button serves directly without ever writing to disk.

5.10 Data Loading

```
@st.cache_data def load_data():     return pd.read_csv('tox21_train_clean.csv'), pd.read_csv('ktest.csv')
```

The @st.cache_data decorator is critical — it tells Streamlit to run this function only ONCE and cache (remember) the result. Without it, the CSV files would be reloaded from disk on every single user interaction. The two files:

- **Training data with SMILES strings and known toxicity labels (0 or 1)**
- **Test/holdout data for evaluating model performance after training**

5.11 Training Tab Logic

When the user clicks 'Train & Evaluate', the following happens in sequence:

1. train_df['smiles'].apply(featurize) — Converts every SMILES in the training set to a 1024-bit fingerprint
2. train_feat.notnull() — Filters out any molecules that failed parsing (invalid SMILES)
3. Same process repeated for test set
4. The selected model is instantiated and trained with model.fit(X_train, y_train)
5. model.predict(X_test) generates predictions on the test set
6. accuracy_score and f1_score compare predictions to actual labels
7. Results stored in st.session_state for use in the Prediction tab
8. 4-panel Matplotlib chart is drawn and displayed

5.12 Prediction Tab Logic

When the user clicks 'Predict':

- The SMILES input is validated with Chem.MolFromSmiles()
- If valid, featurize() converts it to a fingerprint

- model.predict([features]) returns 0 or 1
- model.predict_proba([features]) returns the probability of each class
- The confidence is max(probs) — the probability of the predicted class
- Draw.MolToImage() renders the 2D structure
- All results stored in a 'payload' dictionary
- UI cards display the prediction badge, confidence bar, descriptors, Lipinski check
- build_material_pdf_report(payload) generates the downloadable PDF
- The prediction is appended to history in session_state

5.13 History Tab Logic

The History tab reads from st.session_state['history'] — a list of dictionaries. It displays:

- Total predictions count, how many were Toxic, how many Non-Toxic
- A table of all predictions (SMILES, Prediction, Confidence %)
- Clear History button (calls st.rerun() after clearing)
- Export CSV download button

6. GUI Coding — How the Interface Is Built

6.1 What is GUI Coding?

GUI stands for Graphical User Interface — the visual part of the application that users see and interact with (buttons, text boxes, charts, etc.). In traditional desktop programming, building a GUI requires frameworks like Tkinter, PyQt, or JavaFX with extensive boilerplate code.

Streamlit revolutionized this by allowing Python developers to create full web-based GUIs with simple function calls like st.button(), st.text_input(), and st.tabs(). There is no need to write HTML templates, CSS files, or JavaScript event handlers.

6.2 Layout System

The layout is built using these key Streamlit concepts:

- **Creates horizontal tabs (Training, Prediction, History). Content inside 'with tab1:' appears only when that tab is selected.**
- **Divides the row into two columns with a 2:1 width ratio. Used for placing input alongside buttons, and for metrics side by side.**
- **Content inside 'with st.sidebar:' appears in the collapsible left panel (Lab Console).**
- **Injects raw HTML into the page, enabling custom cards, badges, headers, and the workflow strip.**

6.3 Custom HTML Components

Several visual components are custom HTML:

HTML Component	CSS Class	What It Shows
Header banner	.lab-header	The navy/gold banner at the top with the title
Disclaimer	.disclaimer	The yellow warning box about AI limitations
Workflow strip	.workflow-strip	The step flow: Input → Featurize → Train → Predict → Confidence → PDF
Metric cards	.lab-card + .lab-metric	The glassmorphic cards showing counts and scores
Prediction badge	.badge-toxic / .badge-safe	The glowing Toxic or Non-Toxic badge with pulse animation
Footer	.footer-note	The dashed bottom reminder box

7. Evaluation Metrics — Understanding the Results

7.1 Accuracy

Accuracy = (Correct Predictions) / (Total Predictions). If the model correctly classifies 850 out of 1000 test compounds, accuracy = 0.85 or 85%.

Limitation: If 90% of data is Non-Toxic and the model always predicts Non-Toxic, it achieves 90% accuracy while being completely useless for finding toxic compounds. This is why F1 Score is also reported.

7.2 F1 Score

F1 Score is the harmonic mean of Precision and Recall:

- **Of all compounds predicted Toxic, how many actually are Toxic?**
- **Of all actually Toxic compounds, how many did the model correctly identify?**
- **The balance between these two. A high F1 means the model is good at finding toxic molecules without too many false alarms.**

F1 ranges from 0 (worst) to 1 (perfect). For toxicity prediction, a good F1 score is critical.

7.3 Confusion Matrix

The confusion matrix is the most informative evaluation tool. It is a 2x2 grid:

	Predicted Non-Toxic	Predicted Toxic
Actually Non-Toxic	True Negative (TN) — Correct!	False Positive (FP) — False Alarm
Actually Toxic	False Negative (FN) — Dangerous Miss!	True Positive (TP) — Correct!

In toxicity screening, False Negatives (predicting a toxic compound is safe) are the most dangerous errors. A good model should minimize FN even at the cost of some FP.

8. Complete Application Workflow

Here is the full journey from user input to downloadable report:

Step	What Happens	Code/Technology Involved
1. Start App	User runs 'streamlit run app_r.py' in terminal	Streamlit server launches, browser opens
2. Load Data	CSV files are loaded into Pandas DataFrames	pd.read_csv() with @st.cache_data
3. Select Model	User picks Random Forest, Logistic Regression, or SVM	st.selectbox()
4. Train	User clicks 'Train & Evaluate'	featurize() → model.fit() → metrics
5. Charts Appear	4-panel Matplotlib chart shows evaluation results	plt.subplots() → st.pyplot()
6. Enter SMILES	User types a SMILES string in Prediction tab	st.text_input()
7. Predict	User clicks 'Predict'	featurize() → model.predict() + predict_proba()
8. Molecule Drawn	2D structure image appears	RDKit Draw.MolToImage()
9. Results Shown	Prediction badge, confidence, descriptors, Lipinski	Streamlit UI cards
10. PDF Generated	In-memory PDF built with ReportLab	build_material_pdf_report()
11. Download	User downloads the PDF report	st.download_button()

12. History	Prediction logged to History tab	st.session_state['history']

9. Dataset — Tox21

The Tox21 (Toxicology in the 21st Century) initiative is a US government-sponsored program that created one of the largest publicly available toxicology datasets. It contains over 12,000 compounds tested against 12 different toxicity assays.

For this project, the dataset is pre-cleaned into two CSV files:

- **Training set with 'smiles' column (input) and 'toxicity' column (label: 0=Non-Toxic, 1=Toxic)**
- **Holdout test set with the same format, used to evaluate after training**

The ML model is trained on Morgan fingerprints (1024-bit binary vectors) extracted from the SMILES strings. The 'toxicity' column (0 or 1) is the target label the model learns to predict.

10. How to Run This Project

Step	Command / Action
1. Install dependencies	pip install streamlit pandas numpy matplotlib rdkit scikit-learn reportlab
2. Place data files	Put tox21_train_clean.csv and ktest.csv next to app_r.py
3. Run the app	streamlit run app_r.py
4. Browser opens	Navigate to http://localhost:8501
5. Train a model	Select model from dropdown, click 'Train & Evaluate'
6. Make prediction	Go to Prediction tab, type a SMILES string, click 'Predict'
7. Download report	Click 'Download Report for this Material (PDF)'

11. Important Limitations and Disclaimer

This application includes multiple disclaimer messages for a critical reason: AI/ML predictions in drug safety are probabilistic, not deterministic. The model has been trained on a subset of

toxicological data and cannot account for all possible interactions, metabolic transformations, or biological contexts.

Limitation	Explanation
Not 100% accurate	No ML model achieves perfect prediction — false positives and negatives occur
Dataset-specific	Trained on Tox21 data only — may not generalize to all chemical classes
No metabolism modeling	Does not account for how the body metabolizes and transforms the compound
Binary only	Predicts Toxic/Non-Toxic only — does not specify mechanism or degree of toxicity
Not for clinical use	Must never be used to make decisions about human health or safety
Wet-lab required	All AI predictions must be confirmed through certified laboratory testing

12. Summary — Key Takeaways for Presentation

When presenting this project, highlight these core points:

- **Early-stage drug toxicity prediction using AI, saving time and resources before expensive lab testing**
- **SMILES string in → Toxic/Non-Toxic prediction out with confidence score and full PDF report**
- **SMILES → RDKit parsing → Morgan Fingerprint (1024 bits) → ML model → Prediction**
- **Random Forest (best for fingerprints), Logistic Regression (fast baseline), SVM (maximum margin)**
- **Streamlit converts Python into a full interactive web app with zero traditional web development**
- **RDKit computes real chemical properties (LogP, TPSA, MW, Lipinski rules)**
- **ReportLab generates a downloadable PDF report with all analysis results**
- **Multiple disclaimers ensure users understand predictions are AI estimates, not lab-certified results**

End of Report | AI Drug Toxicity Prediction Lab